

LC 1636: Sort Array by Increasing Frequency

Authored by: Arpit Raj, LNMIIT Jaipur | Date: October 4, 2026

Problem Understanding & Logic

Goal: Sort the array in increasing order based on the frequency of the values. If multiple values have the same frequency, sort them in decreasing order.

Strategy:

1. Use a map to count frequencies.
2. Store each element along with its frequency in a vector<pair<int, int>> (*pair me store krenge*).
3. Apply a custom comparator to sort (*aur fir custom sort*).

Visualizing the Example

1. Initial Array (nums)

2 3 1 3 2

2. Frequency Map Mapping

(1, 1) (2, 2) (3, 2)

Format: (Value, Frequency)

3. Custom Sort Condition

```
if (freq A == freq B) → return Val A > Val B (Descending)
else → return freq A < freq B (Ascending)
```

4. Final Sorted Array

C++ Code Implementation

```
class Solution {
public:
    vector<int> frequencySort(vector<int>& nums) {

        int n=nums.size();
        map<int,int> mp;

        for(int num:nums){
            mp[num]++;
        }

        //eg: 2,3,1,3,2
        //(1,1) , (2,2) ,(3,2)
        //hence pairme store krenge
        //aur fir custom sort

        vector<pair<int,int>> arr;
        for(int num:nums){
            arr.push_back({num,mp[num]});
        }

        sort(arr.begin(),arr.end(),[](auto &a,auto &b){

            if(a.second==b.second){
                return a.first>b.first;
            }

            return a.second<b.second;
        });

        vector<int> ans;
        for(auto it:arr){
            int num=it.first;

            ans.push_back(num);

        }

        return ans;
    }
};
```